

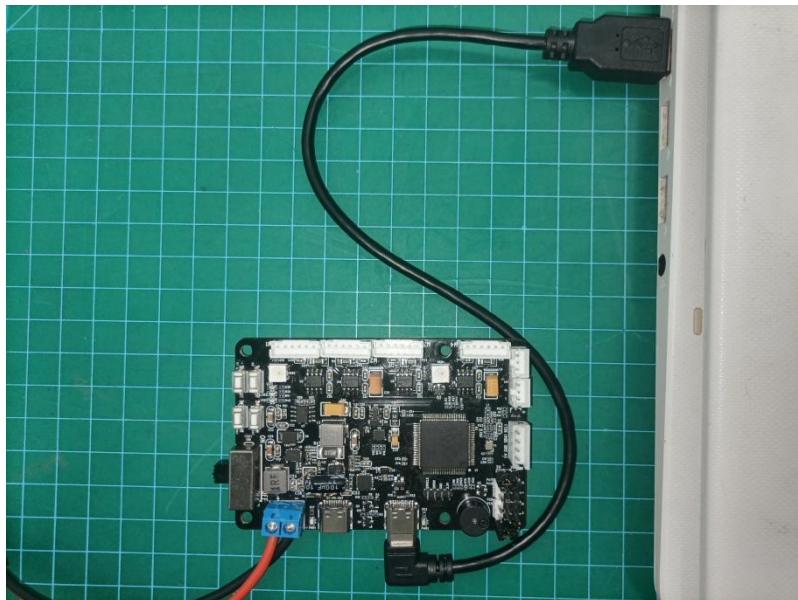
Lesson 12 DC Brushed Motor PID Control and Source Code Analysis

1.Preface

This chapter mainly focuses on PID algorithm control of DC geared motors via the STM32 main control board. Through hardware connections and program downloads, the written code is burnt onto the STM32 main control board via a data connection cable to control the motor's movement. The final section analyzes the source code of the PID control algorithm.

2.Hardware Connection

The encoder motor is connected to M1 port, with a RESET button nearby. The Type-C interface below is for serial port logging, connecting to the computer end to observe the printed data. The physical connection diagram is as follows:



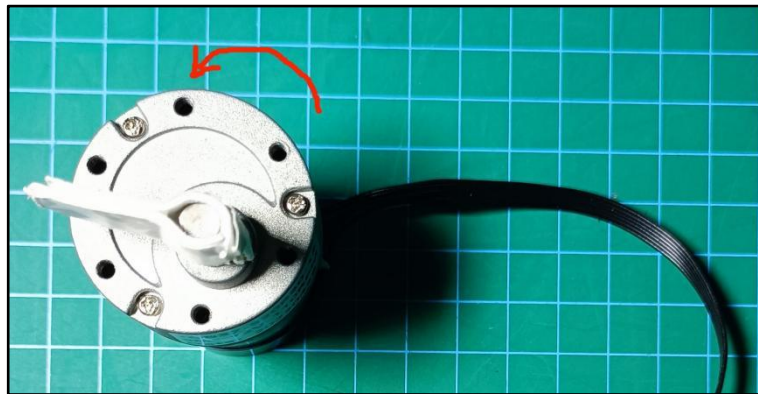
3. Program Download

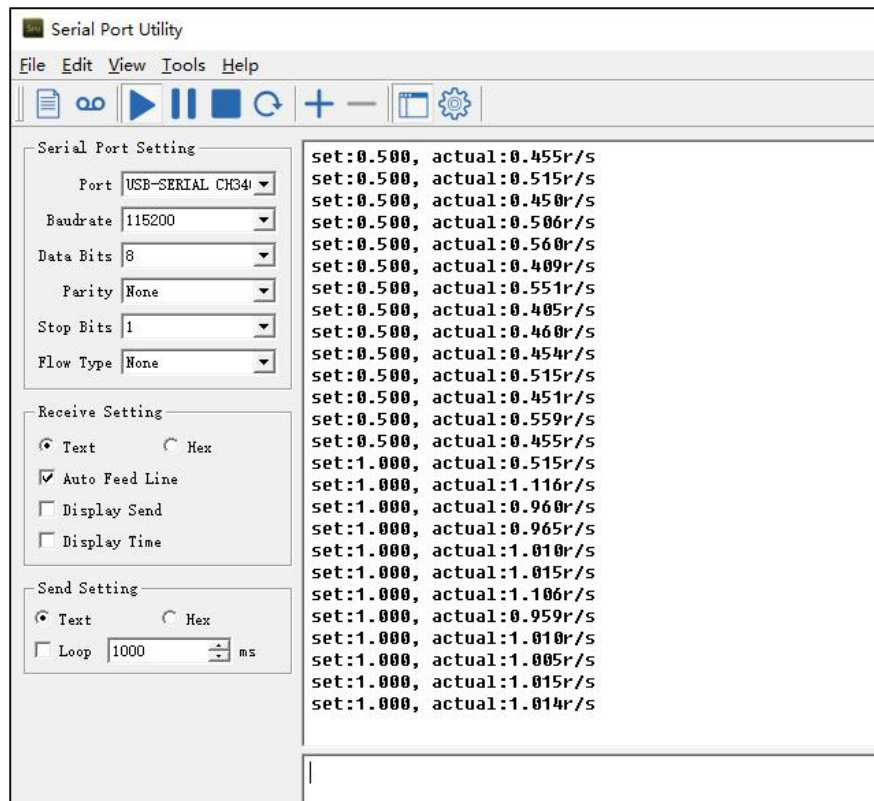
The example routine for this section is called '**EncoderMotor_PID_Demo**'.

After downloading the program (specific downloading instructions can be found in "2. STM32 Development Fundamentals/Lesson 3 Project Compilation and Download"), press the RESET button to reset the microcontroller for the new program to start executing.

4. Program Outcome

After powering on the STM32 control board, the motor runs in both slow and fast states. When viewed from top to bottom, the motor rotates counterclockwise, first at a speed of 1 r/s (rotations per second) for 10 seconds, then at a speed of 0.5 r/s for another 10 seconds, repeating this cycle.





Open the serial assistant (located in "Appendix/3.Software/Serial Assistant"), find the serial port connected to the development board (may not necessarily be COM4, choose the appropriate COM based on actual connection), and configure it as shown below:

Baud Rate = 115200;

Data Bits = 8;

Parity = None;

Stop Bits = 1;

Display Encoding = UTF-8.

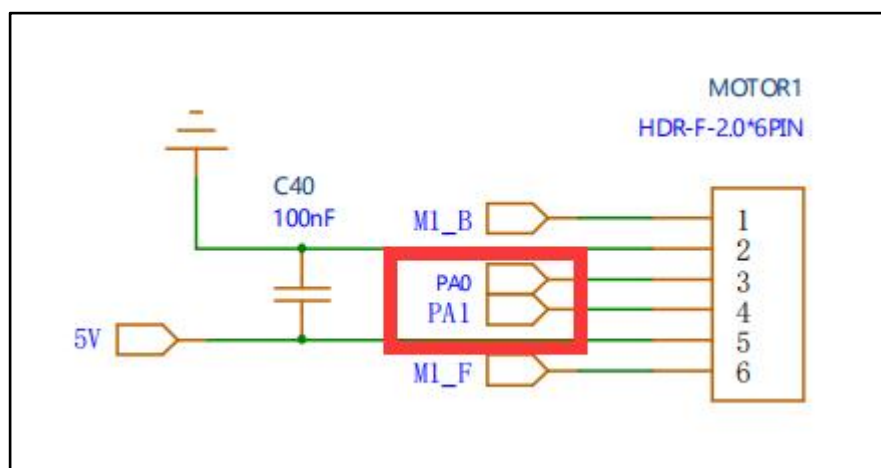
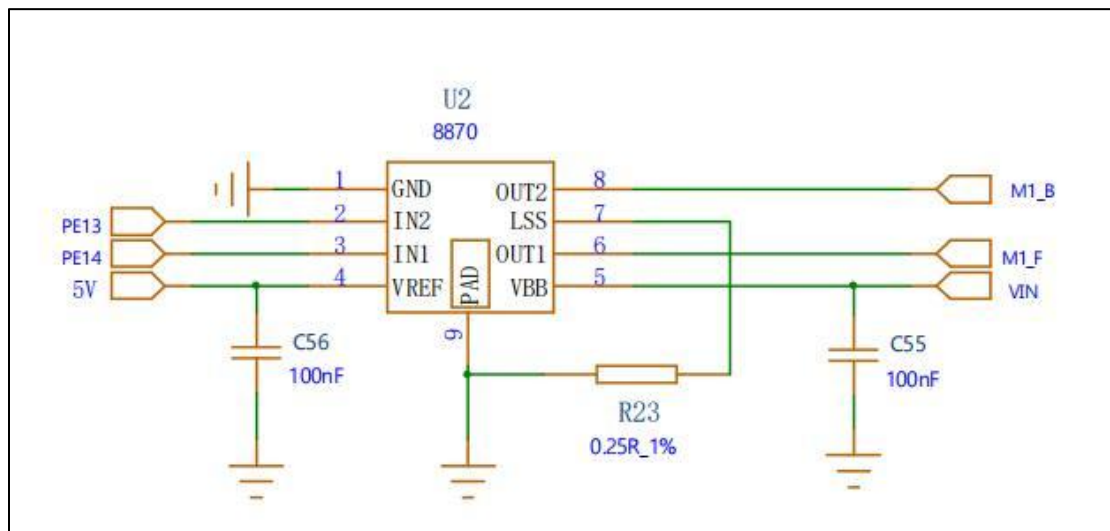
Then, you can open the serial port.

The middle box in the figure displays the data received by the microcontroller through the serial port. The data after 'set' indicates the set speed (r/s), while the data after 'actual' represents the motor's current speed calculated by the

encoder. You may observe a slight deviation between the actual and preset speeds, which is normal.

5. PIN Info Configuration

From the circuit schematic, it can be determined that PE13 and PE14 are used as the motor drive pins for Motor1, while PA0 and PA1 are used as the encoder pins for Motor1. The schematic is as follows:



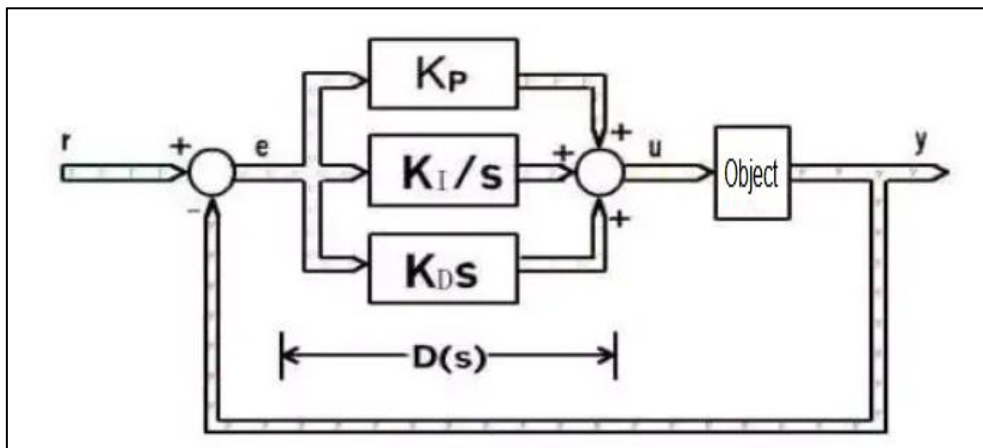
6. Source Code Analysis

The source code for this lesson is located in the same folder as this tutorial document, mainly for analysis of the following files:

Core/Src/main.c,
Hiwonder/motor_porting.c,
Hiwonder/system/app.c, and
Hiwonder/encoder_motor.c.

6.1 PID Principle

The encoder is a sensor used to measure the position of a motor. Its principle involves detecting changes in the sensor's magnetic field during motor rotation to determine the motor's position. As the motor rotates, the encoder outputs a binary code representing the current motor position.



PID control is a commonly used closed-loop control algorithm for controlling the speed or position of a motor. The PID controller consists of three parts:

Proportional (P): The proportional part controls the output based on the difference between the motor position and the target position.

Integral (I): The integral part adjusts the output based on the cumulative difference in position.

Derivative (D): The derivative part adjusts the output based on the rate of change in position.

In motor PID control, the encoder is used to measure the actual motor position. The PID controller calculates based on the difference between the target position and the actual position, and outputs a control signal to control the motor's movement. The PID controller can adjust the control signal based on

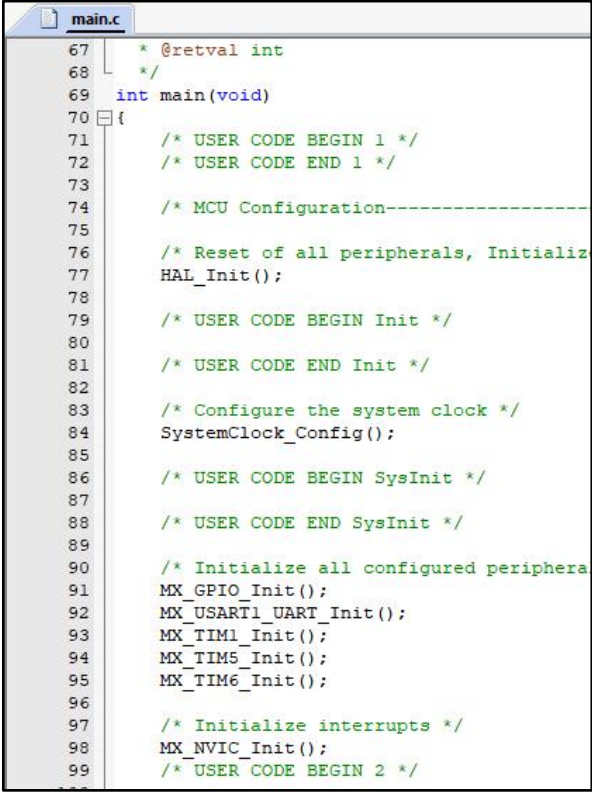
changes in the actual position to better control the motor's movement.

For more information on PID principles, refer to "Lesson 12 RRCLite DC Brushed Motor PID Control and Source Code Analysis/PID Basic Learning."

6.2 Code Analysis

In the main.c file located in the Application/User/Core folder, the main() function serves as the entry point of the project.

- **Basic Configuration Initialization**



```
67  * @retval int
68  */
69  int main(void)
70  {
71      /* USER CODE BEGIN 1 */
72      /* USER CODE END 1 */
73
74      /* MCU Configuration-----
75
76      /* Reset of all peripherals, Initializ
77      HAL_Init();
78
79      /* USER CODE BEGIN Init */
80
81      /* USER CODE END Init */
82
83      /* Configure the system clock */
84      SystemClock_Config();
85
86      /* USER CODE BEGIN SysInit */
87
88      /* USER CODE END SysInit */
89
90      /* Initialize all configured periphera
91      MX_GPIO_Init();
92      MX_USART1_UART_Init();
93      MX_TIM1_Init();
94      MX_TIM5_Init();
95      MX_TIM6_Init();
96
97      /* Initialize interrupts */
98      MX_NVIC_Init();
99      /* USER CODE BEGIN 2 */
```

The diagram illustrates the initialization process including system clock initialization (SystemClock_Config), GPIO initialization (MX_GPIO_Init), UART initialization (MX_USART1_UART_Init), timer 1, 5, and 6 initialization (MX_TIM1_Init, MX_TIM5_Init, MX_TIM6_Init), and interrupt initialization (MX_NVIC_Init). To delve into specific details, you can select the corresponding function and press 'F12' to enter the function body and review the comments provided.

● Motor Initialization

```
102     motor_init();
103
```

The diagram above shows the motor initialization function, which is located in the motor_porting.c file in the Hiwonder folder. The specific function is as depicted in the following diagram:

```
void motors_init(void)
{
    for(int i = 0; i < 4; ++i) {
        motors[i] = LWMEM_CCM_MALLOC(sizeof( EncoderMotorObjectTypeDef));
        encoder_motor_object_init(motors[i]);
        motors[i]->ticks_overflow = 60000;
        motors[i]->ticks_per_circle = MOTOR_DEFAULT_TICKS_PER_CIRCLE;
        motors[i]->rps_limit = MOTOR_DEFAULT_RPS_LIMIT;
        motors[i]->pid_controller.set_point = 0.0f;
        motors[i]->pid_controller.kp = MOTOR_DEFAULT_PID_KP;
        motors[i]->pid_controller.ki = MOTOR_DEFAULT_PID_KI;
        motors[i]->pid_controller.kd = MOTOR_DEFAULT_PID_KD;
    }

    /* Motor 1 */
    motors[0]->set_pulse = motor1_set_pulse;
    __HAL_TIM_SET_COUNTER(&htim1, 0);
    __HAL_TIM_ENABLE(&htim1);
    __HAL_TIM_MOE_ENABLE(&htim1);

    /* encoder */
    __HAL_TIM_SET_COUNTER(&htim5, 0);
    __HAL_TIM_CLEAR_IT(&htim5, TIM_IT_UPDATE);
    __HAL_TIM_ENABLE_IT(&htim5, TIM_IT_UPDATE);
    __HAL_TIM_ENABLE(&htim5);
    HAL_TIM_Encoder_Start(&htim5, TIM_CHANNEL_ALL);

    /* Motor 2 */
    motors[1]->set_pulse = motor2_set_pulse;
    __HAL_TIM_SET_COUNTER(&htim1, 0);
    __HAL_TIM_ENABLE(&htim1);
    __HAL_TIM_MOE_ENABLE(&htim1);

    // Speed update timer
    __HAL_TIM_SET_COUNTER(&htim7, 0);
    __HAL_TIM_CLEAR_IT(&htim7, TIM_IT_UPDATE);
    __HAL_TIM_ENABLE_IT(&htim7, TIM_IT_UPDATE);
    __HAL_TIM_ENABLE(&htim7);
}
```

Initialize the motor structure, then set the parameters corresponding to the motor structure (here, parameters need to be set according to the motor model; simply find the corresponding motor parameters in the encoder_motor.h file and replace the parameters here).

Start the motor control timer TIM1, set and start the encoder timer TIM5, then configure and start the speed calculation timer TIM7.

● Motor Loop Control

```
Within the loop {
    Set the motor speed to 1.0 r/s;
    Loop 10 times, printing data each time, with a delay of 100ms between iterations;
    Set the motor speed to 0.5 r/s;
    Loop 10 times, printing data each time, with a delay of 100ms between iterations;
```

```
}
```

The specific program is as below:

```
for(;;) {
    //encoder_motor_set_speed() Set rotation speed of the servo with PID algorithm. Parameter 1: motor object; parameter 2: set motor speed (r/s). After compiling,
    encoder_motor_set_speed(motors[0], 1.0); /* set target rotation speed to 1.0 */
    for(int i = 0; i < 100; ++i) {
        printf("set:%0.3f, actual:%0.3f/s\r\n", motors[0]->pid_controller.set_point, motors[0]->rps); // print the set speed and measured speed
        osDelay(100); // delay for 100ms
    }
    encoder_motor_set_speed(motors[0], 0.5); /* set target rotation speed to 0.5 */
    for(int i = 0; i < 100; ++i) {
        printf("set:%0.3f, actual:%0.3f/s\r\n", motors[0]->pid_controller.set_point, motors[0]->rps); // print the set speed and measured speed
        osDelay(100); // delay for 100ms
    }
}
```

The printed data: the data after 'set' represents the set speed (r/s), while the data after 'actual' represents the current motor speed calculated by the encoder.

● Timer 7

The function of Timer 7 is to calculate motor speed and update PID parameters. It generates an update interrupt every 10ms. The update interrupt function is as follows (located in the motor_porting.c file in the Hiwonder folder):

```
void TIM7_IRQHandler(void)
{
    /* USER CODE BEGIN TIM7_IRQn 0 */
    extern EncoderMotorObjectTypeDef *motors[4];
    if(__HAL_TIM_GET_FLAG(&tim7, TIM_FLAG_UPDATE) != RESET) {
        __HAL_TIM_CLEAR_FLAG(&tim7, TIM_FLAG_UPDATE);
        encoder_update(motors[0], 0.01, __HAL_TIM_GET_COUNTER(&tim5));
        encoder_update(motors[1], 0.01, __HAL_TIM_GET_COUNTER(&tim2));
        encoder_update(motors[2], 0.01, 60000 - __HAL_TIM_GET_COUNTER(&tim4));
        encoder_update(motors[3], 0.01, __HAL_TIM_GET_COUNTER(&tim3));
        for(int i = 0; i < 4; ++i) {
            encoder_motor_control(motors[i], 0.01);
        }
    }
    /* USER CODE END TIM7_IRQn 0 */
    /* USER CODE BEGIN TIM7_IRQn 1 */

    /* USER CODE END TIM7_IRQn 1 */
}
```

encoder_update() is the function responsible for calculating the updated motor speed based on the encoder count. Its specific implementation is as follows (located in the encoder_motor.c file in the Hiwonder folder):

```
/**
 * @brief Encoder motor speed measurement update
 * @details
 * @param self Encoder motor object pointer
 * @param period The time interval (update cycle) between the current update and the last update, in seconds
 * @param counter Current count value of encoder
 * @retval None.
 */
void encoder_update(EncoderMotorObjectTypeDef *self, float period, int64_t counter)
{
    counter = counter + self->overflow_num * self->ticks_overflow; /* The total count value is 60000, based on the actual timer overflow value set */
    int delta_count = counter - self->counter;
    self->counter = counter; /* Store new count values */
    self->tps = (float)delta_count / period * 0.9f + self->tps * 0.1f; /* Calculate pulse frequency */
    self->rps = self->tps / self->ticks_per_circle; /* Calculate the speed unit rps, revolutions per second */
}
```

encoder_motor_control() is the function for updating the encoder motor speed control. It first updates the PID parameters, corrects the output PWM, and

generates a new PWM signal. Its specific implementation is as follows (located in the encoder_motor.c file in the Hiwonder folder)

```
/**
 * @brief Encoder motor speed control task
 * @details The encoder motor speed PID control task requires timed specification to complete PID control updates
 * @param self Encoder motor object pointer
 * @param period The time interval (update cycle) between the current update and the last update, in seconds
 * @retval None.
 */
void encoder_motor_control(EncoderMotorObjectTypeDef *self, float period)
{
    float pulse = 0;
    pid_controller_update(&self->pid_controller, self->rps, period); /* Update PID controller */
    pulse = self->current_pulse + self->pid_controller.output; /* Calculate new PWM values */

    /* Limit the output PWM value according to the timer setting. In this example, the duty cycle of the timer set from 0-100 corresponds to 0-1000 */
    pulse = pulse > 1000 ? 1000 : pulse;
    pulse = pulse < -1000 ? -1000 : pulse;

    self->set_pulse(self, pulse > -250 && pulse < 250 ? 0 : pulse); /* Set a new PWM value and limit the minimum PWM value. If the PWM is too small, the motor
    self->current_pulse = pulse; /* Record new PWM values */
}
```

The PID controller update function, pid_controller_update(), is the core of PID control, specifically speed PID control.

It first calculates the error 'err', then calculates the proportional, integral, and derivative values based on time.

Next, it calculates the correction value for the output, and finally updates the error parameters.

The specific code implementation is shown in the diagram below.

```
15 void pid_controller_update(PID_ControllerTypeDef *self, float actual, float time_delta) {
16     float err = self->set_point - actual;
17     float proportion = err - self->previous_0_err;
18
19     float integral = err * time_delta;
20     float derivative = (err - 2 * self->previous_1_err + self->previous_0_err) / time_delta;
21
22     self->output = (self->kp * err) + (self->ki * integral) + (self->kd * derivative);
23     self->previous_1_err = self->previous_0_err;
24     self->previous_0_err = err;
25 }
26
```

In this context:

- 'err' represents the error between the current target value and the actual value.
- 'proportion' is the difference between the current error and the previous error, used for calculating the derivative.
- 'integral' accumulates the time without friction, used in combination with the derivative for calculation.
- 'derivative' represents the derivative parameter.
- Finally, the function stores the result in the 'output' field of the PID_ControllerTypeDef structure."